

Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №9

РІЗНІ ВИДИ ВЗАЄМОДІЇ ДОДАТКІВ: CLIENT-SERVER, PEER-TO-PEER, SERVICE-ORIENTED ARCHITECTURE

Виконав
студент групи ІА–22:
Артамонов Дмитро Вікторович
Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Хід роботи.....3
Короткі теоретичні відомості 3
Реалізація не менше 3-х класів відповідно до обраної теми.....7
Реалізація архітектури за обраною темою 11
Висновок.....27

Тема: Різні види взаємодії додатків: client-server, peer-to-peer, service-oriented architecture.

Мета: Ознайомитись з різними видами взаємодії. Реалізувати функціонал для роботи в розподіленому оточенні (логіку роботи).

Хід роботи

..15 E-mail клієнт (singleton, builder, decorator, template method, interpreter, SOA)

Поштовий клієнт повинен нагадувати функціонал поштових програм Mozilla Thunderbird, The Bat і т.д. Він повинен сприймати і коректно обробляти

---

pop3/smtp/imap протоколи, мати функції автонастройки основних поштових провайдерів для України (gmail, ukr.net, i.ua), розділяти повідомлення на папки/категорії/важливість, зберігати чернетки незавершених повідомлень, прикріплювати і обробляти прикріплені файли.

Короткі теоретичні відомості

Архітектура додатків визначає спосіб організації компонентів програмного забезпечення та їх взаємодії для досягнення функціональних і нефункціональних вимог системи. Вона задає структуру, правила й стандарти для побудови додатків, забезпечуючи масштабованість, ефективність і легкість супроводу.

Основні принципи архітектури додатків:

1. **Розподіл обов'язків**: різні компоненти виконують конкретні функції, що підвищує модульність і зручність супроводу.
2. **Взаємодія через стандартизовані інтерфейси**: забезпечує сумісність між компонентами незалежно від платформи чи технології.
3. **Масштабованість**: архітектура має підтримувати адаптацію до зростання навантаження.

4. **Незалежність компонентів**: мінімізація залежностей між частинами системи для спрощення оновлень і розширення.

Типи архітектур відрізняються способами взаємодії компонентів:

- У **централізованих** архітектурах є чітко визначений центр управління.
- У **розподілених** архітектурах компоненти взаємодіють через мережу, що дає більшу гнучкість і стійкість до відмов.
- **Гібридні** архітектури комбінують підходи для задоволення специфічних потреб.

Вибір архітектури залежить від вимог проєкту, таких як масштабованість, надійність, продуктивність і простота супроводу. Основні моделі, такі як клієнт-сервер, peer-to-peer і сервіс-орієнтована архітектура, реалізують ці принципи з урахуванням різних сценаріїв використання.

Клієнт-серверні додатки

Клієнт-серверні додатки – це розподілені системи, що складаються з клієнтів (для взаємодії з користувачем) та серверів (для обробки й зберігання даних). Розрізняють **тонкі** та **товсті** клієнти.

Тонкий клієнт передає більшість обчислень на сервер, залишаючи на собі лише візуальне відображення даних. Це зменшує вимоги до клієнтських пристроїв, але збільшує навантаження на сервер. Така модель актуальна для захищених сценаріїв (захист даних від зламу) або для уникнення конфліктів при множинному доступі до даних.

Товстий клієнт виконує більшість обчислень локально, розвантажуючи сервер, який виступає лише як точка доступу до ресурсів (наприклад, бази даних).

Для взаємодії використовується модель «підписка/видача», де клієнти підписуються на події, а сервер їх сповіщає. Це зручно для оновлення даних, але створює навантаження на сервер і канали зв’язку.

Зазвичай клієнт-серверна структура організовується на 3 рівні:

- **Клієнтська частина**: відображення даних, обробка дій користувача, зв’язок із сервером.
- **Загальна частина (middleware)**: спільні класи й компоненти.
- **Серверна частина**: бізнес-логіка, зберігання та обмін даними.

Додатки типу «peer-to-peer»

Модель взаємодії додатків типу peer - to - peer має на увазі рівноправ'я клієнтських програм і відсутність серверної програми. Усі клієнтські програми контактують між собою для виконання спільних цілей. У таких мережах найчастіше виникають наступні проблеми: синхронізація даних, пошук клієнтських застосувань.

Для пошуку клієнтських застосувань може використовуватися одна загальна адреса, що містить набір адрес підключених клієнтів. Це в деякому роді нагадує сервер. Проте сервер є лише центром сходження безлічі клієнтів, а адреси можуть зберігатися в самих клієнтських застосуваннях. Це називається структуровані однорангові мережі.

Для вирішення проблеми синхронізації даних використовуються традиційні підходи: можливий варіант договору про набір для синхронізації (negotiation), або визначення по деякому алгоритму порівняння даних (hash- алгоритми і інші). Як правило, peer - to - peer застосування є звичайними застосуваннями з продуманою інфраструктурою обміну повідомленнями між клієнтами. Для таких випадків як правило розробляються спеціальні формати і протоколи обміну для структуризації діалогу між клієнтами.

Сервіс-орієнтована архітектура

Сервіс-орієнтована архітектура (SOA, service-oriented architecture) — це підхід до розробки програмного забезпечення, який передбачає використання розподілених, слабо пов’язаних компонентів із стандартизованими інтерфейсами для взаємодії через стандартизовані протоколи. SOA дозволяє створювати масштабовані, керовані системи, незалежні від платформи та інструментів розробки. Програмні комплекси, побудовані за SOA, зазвичай реалізуються як набір веб-служб, що використовують протоколи SOAP або REST. Інтерфейси компонентів інкапсулюють деталі реалізації, що забезпечує можливість багаторазового використання та легкість інтеграції.

SaaS (Software as a Service) — це бізнес-модель, у якій постачальник розробляє веб-додаток, самостійно ним керує та надає клієнтам доступ через Інтернет. Основні риси SaaS:

- Відсутність витрат на встановлення, оновлення й підтримку ПЗ та обладнання.
- Один додаток може обслуговувати кілька клієнтів.
- Оплата здійснюється як абонентська плата або залежить від обсягу використання.
- Оновлення та техпідтримка включені у вартість.
- Оновлення відбуваються автоматично та прозоро для клієнтів.

SaaS дозволяє клієнтам зменшити витрати, адже замість купівлі програмного забезпечення й апаратного забезпечення для його розгортання, вони орендують доступ до ПЗ. Це робить модель привабливою для тих, хто потребує гнучкості. При необхідності можна тимчасово припинити використання програми та оплати. Для розробників SaaS знижує витрати на підтримку та впровадження ПЗ, а також допомагає боротися з неліцензійним використанням, оскільки доступ до додатку здійснюється тільки через Інтернет.

Реалізація SaaS базується на SOA та зазвичай є stateless (без стану). Для авторизації користувачів використовуються токени, які видаються після успішного входу. Зберігання даних і обробка запитів виконуються в хмарі, що забезпечує гнучкість і масштабованість систем.

Мікро-сервісна архітектура.

Архітектура мікрослужб — це підхід до розробки додатків, де система складається з невеликих автономних служб. Кожна служба виконується у власному процесі та взаємодіє з іншими через стандартизовані протоколи, такі як HTTP/HTTPS, WebSockets чи AMQP. Служби створюються для виконання

конкретних бізнес-завдань у межах обмеженого контексту, розробляються незалежно та можуть бути розгорнуті автономно. Мікрослужби забезпечують гнучкість і масштабованість у довгостроковій перспективі. Вони дозволяють розробляти, оновлювати й масштабувати систему частинами, що спрощує підтримку складних програм. Завдяки автономності, кожна мікрослужба може використовувати власну технологію, базу даних або мову програмування.

Переваги:

- **Масштабованість**: кожна служба масштабується незалежно.
- **Гнучкість**: використання різних технологій для різних завдань.
- **Стійкість**: збої в одній службі не впливають на роботу інших.
- **Незалежна розробка**: команди працюють над різними службами паралельно.

Виклики:

- **Управління складністю**: потрібні інструменти для моніторингу, оркестрації та забезпечення безпеки.
- **Взаємодія між службами**: важливо забезпечити надійність і швидкість передачі даних.
- **Розподілена природа**: ускладнюється тестування та підтримка узгодженості даних.

Архітектура мікрослужб добре підходить для масштабних і динамічних проєктів, де потрібна швидка адаптація до змін. Вона часто застосовується у хмарних середовищах із використанням таких інструментів, як Docker, Kubernetes, RabbitMQ та API-шлюзи, що допомагають автоматизувати розгортання й управління.

Реалізація не менше 3-х класів відповідно до обраної теми

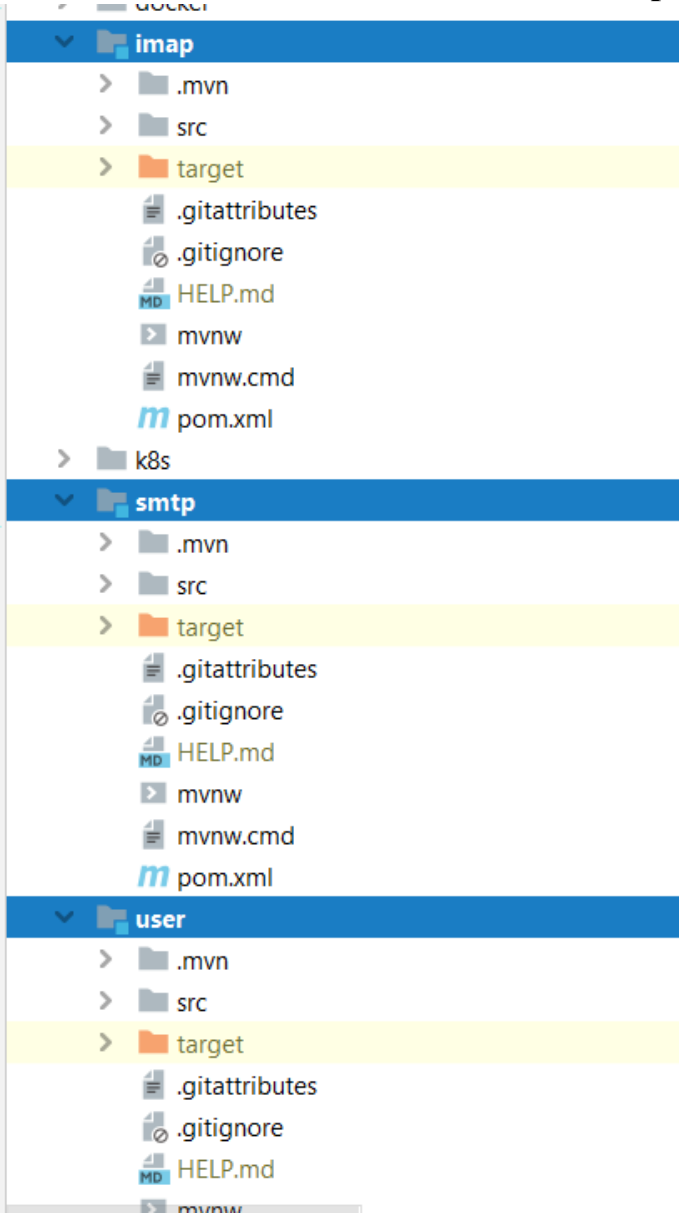


Рисунок 1 Реалізована структура проєкту

У ході лабораторної роботи була реалізована система поштового клієнта з використанням архітектури Service-Oriented Architecture (SOA), яка забезпечує модульну взаємодію між компонентами програми через стандартизовані інтерфейси. Основним завданням було створення функціонального додатка, який підтримує обробку протоколів SMTP і IMAP, автоматичне налаштування основних поштових провайдерів України, сортування повідомлень за папками, категоріями та рівнем важливості, а також збереження чернеток незавершених повідомлень. У результаті виконання роботи система стала гнучкою і масштабованою, що дозволяє легко додавати нові сервіси або оновлювати існуючі.

1. Розроблено структуру проєкту:
    - Кожен модуль розбититй на логічні пакети: repository, service, client, common, config, domain, web та інші, що забезпечує зручність масштабування та підтримки коду.
    - Для кожного сервісу створено власний REST API, який відповідає за взаємодію із зовнішніми компонентами.
2. Реалізовано архітектуру SOA:
    - DraftService - відповідає за збереження електронних листів як чернеток.
    - ImapService - відповідає за роботу з електронною поштою через IMAP-протокол, включаючи читання, переміщення, створення папок і отримання поштових скриньок.
    - EmailService - відповідає за надсилання електронних листів, налаштування поштових скриньок, перевірку облікових даних і оновлення токенів доступу для Gmail.
    - GmailSendStrategy - відповідальний за відправлення електронних листів через Gmail за допомогою протоколу SMTP із використанням OAuth2 для автентифікації.
    - UkrNetSendStrategy - відповідає за відправлення електронних листів через протокол SMTP для поштових скриньок, що використовують UkrNet, з автентифікацією за допомогою логіна та пароля.
    - UserServiceImpl - відповідає за реєстрацію користувачів, аутентифікацію з JWT токенами, валідацію токенів, додавання поштових скриньок та взаємодію з клієнтами SMTP і IMAP для управління поштовими акаунтами користувачів.

3. Реалізовано REST API для взаємодії:

Для кожного сервісу створено окремий контролер:

Опис ендпойнтів для EmailController:

- **GET /api/v1/email:** Вітає користувача та перелічує доступні ендпойнти системи (/send, /config).
- **POST /api/v1/email/send:** Надсилає електронний лист.
- **POST /api/v1/email/config:** Додає або змінює конфігурацію поштової скриньки.

Опис ендпойнтів для AuthController:

- **GET /api/v1/auth:**
  Вітає користувача та перелічує доступні ендпойнти системи (/create, /sign-in, /validate-token/{token}).
- **POST /api/v1/auth:**
  Створює нового користувача. Приймає дані у форматі UserCreationDto та повертає інформацію про створеного користувача.
- **POST /api/v1/auth/sign-in:**
  Виконує автентифікацію користувача. Приймає логін і пароль у форматі AuthenticationRequest та повертає токен доступу у вигляді AuthenticationResponse.
- **GET /api/v1/auth/validate-token/{token}:**
  Перевіряє дійсність токена доступу. Повертає інформацію про користувача, якщо токен валідний.

Опис ендпойнтів для UserController:

- **GET /api/v1/users:**
  Обробляє зворотний виклик від Google із параметром code. Виводить авторизаційний код для подальшої обробки.
- **GET /api/v1/users/mailbox:**
  Отримує список поштових скриньок, пов’язаних із поточним користувачем, визначеним через Principal.
- **POST /api/v1/users/add-account:**
  Додає нову поштову скриньку для користувача. Приймає дані у форматі MailBoxCreation, повертає інформацію про створену поштову скриньку.

Опис ендпойнтів для FolderController:

- **POST /api/v1/folder/create/{account}/{folderName}:**
  Створює нову папку для зазначеного облікового запису. Приймає назву облікового запису, ім’я папки та автентифікованого користувача через Principal.
- **POST /api/v1/folder/move/{account}/{msgnum}:**
  Переміщує електронний лист із зазначеної папки в іншу. Приймає обліковий запис, номер повідомлення (msgnum), назву вихідної та цільової папок через folderNameMap, а також користувача через Principal.
- **POST /api/v1/folder/add-to-draft:**
  Зберігає електронний лист як чернетку. Приймає дані листа у форматі EmailCreationDto і зберігає їх у папку чернеток користувача.

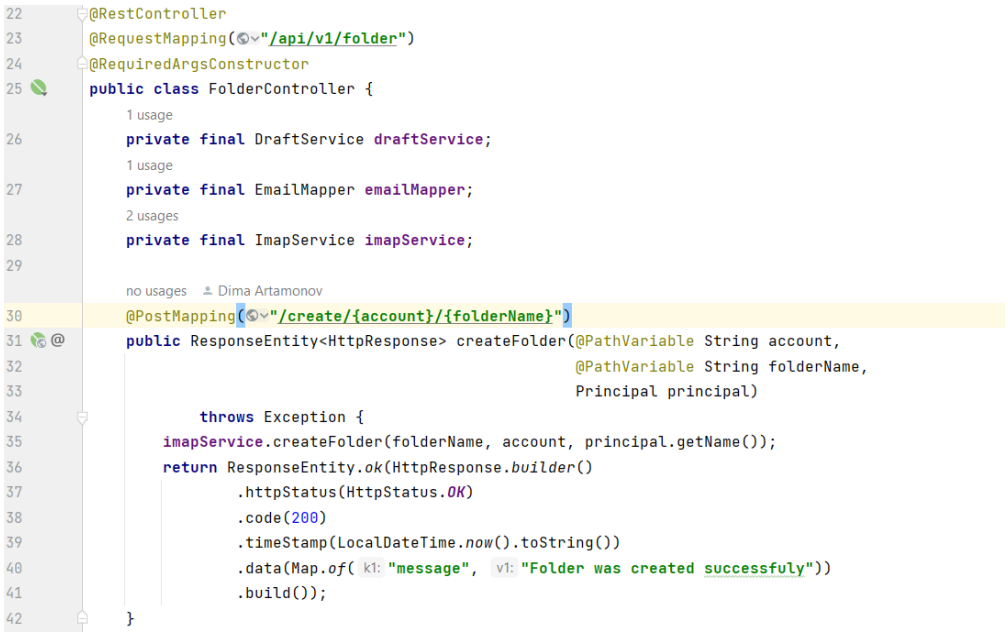
Опис ендпойнтів для ImapController:

- **POST /api/v1/read/{account}:**
  Зчитує електронні листи з вказаної папки для зазначеного облікового запису. Приймає назву облікового запису та назву папки через folderNameMap, повертає список листів.
- **POST /api/v1/read/{account}/{msgnum}:**
  Отримує конкретний електронний лист за номером повідомлення (msgnum) з вказаної папки. Приймає обліковий запис і назву папки через folderNameMap, повертає вміст листа.
- **GET /api/v1/read/mailbox:**
  Повертає список поштових скриньок, пов’язаних із автентифікованим користувачем.

4. Перевірено функціоналу системи та тестування**:** Функціонал системи перевірено, що підтвердило її коректну роботу та відповідність вимогам. Завдяки впровадженню архітектури SOA поштовий клієнт став більш гнучким, масштабованим та придатним для подальшого розвитку. Нова структура дозволяє легко інтегрувати додаткові сервіси й функції без впливу на існуючі модулі.

Реалізація архітектури за обраною темою

FolderController:



```
@RestController
@RequestMapping("/api/v1/folder")
@RequiredArgsConstructor
public class FolderController {
    1 usage
    private final DraftService draftService;
    1 usage
    private final EmailMapper emailMapper;
    2 usages
    private final ImapService imapService;

    no usages  Dima Artamonov
    @PostMapping("/create/{account}/{folderName}")
    public ResponseEntity<HttpResponse> createFolder(@PathVariable String account,
                                                     @PathVariable String folderName,
                                                     Principal principal)
            throws Exception {
        imapService.createFolder(folderName, account, principal.getName());
        return ResponseEntity.ok(HttpResponse.builder()
                .httpStatus(HttpStatus.OK)
                .code(200)
                .timeStamp(LocalDateTime.now().toString())
                .data(Map.of( k1: "message",  v1: "Folder was created successfuly"))
                .build());
    }
```

Рисунок 2 Клас FolderController

Цей клас є контролером, який забезпечує взаємодію між клієнтськими запитами та сервісами для роботи з папками та чернетками у поштовому клієнті. Він визначає API для створення папок, переміщення листів між папками та збереження листів як чернеток, делегуючи виконання бізнес-логіки відповідним сервісам.

ImapController:

```java
@RestController
@RequestMapping("/api/v1/read")
@RequiredArgsConstructor
public class ImapController {
    1 usage
    private final EmailMapper emailMapper;
    3 usages
    private final ImapService imapService;

    no usages  Dima Artamonov
    @PostMapping("/{account}")
    public ResponseEntity<HttpResponse> readEmails(@PathVariable String account,
                                                   @RequestBody Map<String, String> folderNameMap,
                                                   Principal principal)
            throws Exception {
        String folderName = folderNameMap.get("folderName");
        var messages = imapService.getEmails(account, folderName, principal.getName());
        return ResponseEntity.ok(HttpResponse.builder()
                .httpStatus(HttpStatus.OK)
                .code(200)
                .timeStamp(LocalDateTime.now().toString())
                .path(String.format("/api/v1/read/%s/%s", account, folderName))
                .data(Map.of( k1: "List of emails", Arrays.stream(messages).map((temp) -> {
                    try {
                        return emailMapper.toReceivedEmail(temp);
                    } catch (MessagingException e) {
                        throw new RuntimeException(e);
                    }
                }).toList()))
                .build());
    }
```

Рисунок 3 Клас ImapController

```
    @PostMapping("/{account}/{msgnum}")
    public ResponseEntity<HttpResponse> getSpecificEmail(@PathVariable String account,
                                                         @RequestBody
                                                         Map<String, String> folderNameMap,
                                                         @PathVariable int msgnum,
                                                         Principal principal)
            throws Exception {
        String folderName = folderNameMap.get("folderName");
        var email = imapService.getSpecificEmail(account, folderName, msgnum, principal.getName());
        return ResponseEntity.ok(HttpResponse.builder()
                .httpStatus(HttpStatus.OK)
                .code(200)
                .path(String.format("/api/v1/read/%s/%s/%s", account, folderName, msgnum))
                .data(Map.of( k1: "email", email))
                .timeStamp(LocalDateTime.now().toString())
                .build());
    }

    no usages  Dima Artamonov
    @GetMapping("/mailbox")
    public ResponseEntity<List<MailBoxDto>> getMailBoxes(Principal principal) {
        return ResponseEntity.ok(imapService.getMailBoxes(principal.getName()));
    }
}
```

Рисунок 4 Клас ImapController

Цей клас є контролером, який забезпечує API для взаємодії з поштовим сервером через протокол IMAP. Він дозволяє читати електронні листи, отримувати конкретний лист та отримувати список поштових скриньок, делегуючи виконання бізнес-логіки сервісам.

EmailController:

```java
@RestController
@RequestMapping("/api/v1/email")
@RequiredArgsConstructor
public class EmailController {
    2 usages
    private final EmailService emailService;
    1 usage
    private final EmailMapper emailMapper;

    no usages  Dima Artamonov
    @PostMapping
    public ResponseEntity<HttpResponse> sendEmail(
            @Valid @RequestBody EmailCreationDto emailCreationDto, Principal principal)
            throws UnsupportedEncodingException, MessagingException {
        System.out.println(principal.getName());
        var email =
                emailService.sendEmail(emailMapper.toEmail(emailCreationDto), principal.getName());
        return ResponseEntity.ok(new HttpResponseEmailBuilder().build());
    }

    no usages  Dima Artamonov
    @PostMapping("/config")
    public ResponseEntity<MailBox> setEmailConfig(@RequestBody MailBox mailBox)
            throws MessagingException, IOException {
        return ResponseEntity.ok(emailService.addEmailConfiguration(mailBox));
    }

}
```

Рисунок 5 Клас EmailController

Цей клас є контролером, який забезпечує API для роботи з електронною поштою. Він дозволяє надсилати електронні листи та налаштовувати поштову скриньку, делегуючи виконання бізнес-логіки відповідним сервісам.

UserController:



Рисунок 6 Клас UserController

Цей клас є контролером, який забезпечує API для управління поштовими скриньками користувачів. Він дозволяє отримувати список поштових скриньок, додавати нову поштову скриньку та обробляти колбеки для авторизації через сторонні сервіси. Виконання бізнес-логіки делегується відповідним сервісам.

AuthController:



Рисунок 7 Клас AuthController

Цей клас є контролером, який забезпечує API для реєстрації користувачів, авторизації та валідації токенів. Він дозволяє створювати нового користувача, здійснювати вхід за допомогою логіну та пароля, а також перевіряти дійсність токенів. Бізнес-логіка реалізована в сервісах, а маппінг даних здійснюється через відповідні мапери.

UserService:

```java
@Service
@RequiredArgsConstructor
public class UserServiceImpl implements UserService {

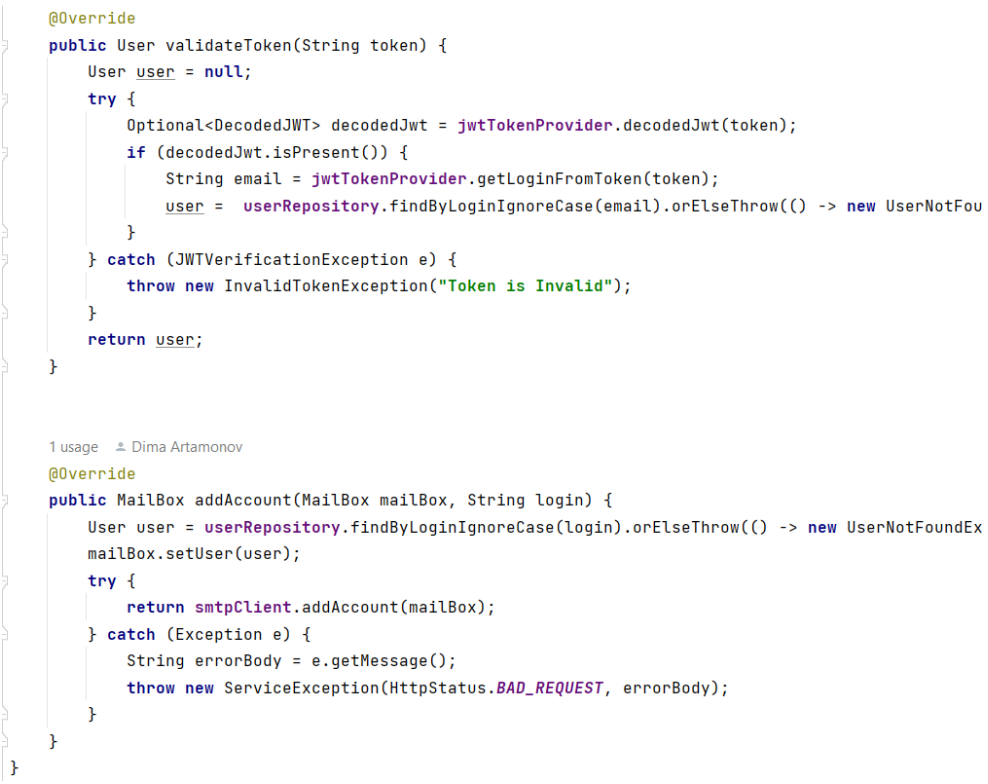
    6 usages
    private final UserRepository userRepository;
    1 usage
    private final SmtpClient smtpClient;
    1 usage
    private final ImapClient imapClient;
    2 usages
    private final PasswordEncoder encoder;
    5 usages
    private final JwtTokenProvider jwtTokenProvider;

    1 usage  Dima Artamonov
    @Override
    public User saveUser(User user) throws IOException {
        if (userRepository.existsByLogin(user.getLogin())) {
            throw new UserAlreadyExistsException("User already exists");
        }
        user.setEnable(false);
        user.setPassword(encoder.encode(user.getPassword()));
        userRepository.save(user);
        return user;
    }

    @Override
    @Transactional
    public Optional<DecodedJWT> signIn(String login, String password) {
        var user = userRepository.findByLoginIgnoreCase(login).orElseThrow(() -> new UserNotFoundExce
                "User with login %s doesn`t exist".formatted(login)
        ));
        if (!encoder.matches(password, user.getPassword())) {
            throw new InvalidPasswordException("Invalid password");
        }
        var token = jwtTokenProvider.generateToken(user);
        return jwtTokenProvider.decodedJwt(token);
    }

    1 usage  Dima Artamonov
    @Override
    public List<MailBoxDto> getMailBoxes(String login) {
        String decodedJWT = jwtTokenProvider.generateToken(userRepository.findByLoginIgnoreCase(logir
        String authorizationHeader = "Bearer " + decodedJWT;
        return imapClient.getMailBoxes(authorizationHeader);
    }
```

Рисунок 8 Клас UserService



```
    @Override
    public User validateToken(String token) {
        User user = null;
        try {
            Optional<DecodedJWT> decodedJwt = jwtTokenProvider.decodedJwt(token);
            if (decodedJwt.isPresent()) {
                String email = jwtTokenProvider.getLoginFromToken(token);
                user =  userRepository.findByLoginIgnoreCase(email).orElseThrow(() -> new UserNotFou
            }
        } catch (JWTVerificationException e) {
            throw new InvalidTokenException("Token is Invalid");
        }
        return user;
    }


    1 usage  Dima Artamonov
    @Override
    public MailBox addAccount(MailBox mailBox, String login) {
        User user = userRepository.findByLoginIgnoreCase(login).orElseThrow(() -> new UserNotFoundEx
        mailBox.setUser(user);
        try {
            return smtpClient.addAccount(mailBox);
        } catch (Exception e) {
            String errorBody = e.getMessage();
            throw new ServiceException(HttpStatus.BAD_REQUEST, errorBody);
        }
    }
}
```

Рисунок 9 Клас UserService

Цей клас реалізує функціонал для управління користувачами в системі. Він відповідає за реєстрацію нових користувачів, автентифікацію через логін та пароль, перевірку валідності токенів, взаємодію з поштовими сервісами для роботи з поштовими скриньками, а також за додавання облікових записів користувачів. Клас забезпечує обробку винятків і гарантує безпечне зберігання паролів та управління токенами через інтеграцію з компонентами безпеки.

EmailServiceImpl:

```java
@RequiredArgsConstructor
public class EmailServiceImpl implements EmailService {
    1 usage
    private static final HttpTransport HTTP_TRANSPORT = new NetHttpTransport();
    1 usage
    private static final JsonFactory JSON_FACTORY = new GsonFactory();
    5 usages
    private final MailBoxRepository mailBoxRepository;
    1 usage
    private final List<SendStrategy> listOfStrategies;
    2 usages
    @Value("${GOOGLE_CLIENT_ID}")
    private String googleClientId;

    2 usages
    @Value("${GOOGLE_CLIENT_SECRET}")
    private String googleClientSecret;
    1 usage
    @Value("http://209.38.112.145.nip.io:9090/api/v1/users")
    private String googleCallBackUri;

    3 usages
    private SendStrategy sendStrategy;

    2 usages  Dima Artamonov
    @Override
    public Email sendEmail(Email email, String login)
            throws MessagingException, UnsupportedEncodingException {
        MailBox mailBox = findMailBox(email.getSenderEmail(), login);
        specifyStrategyByMailBox(mailBox);
```

```java
    @Override
    public MailBox addEmailConfiguration(MailBox mailBox) throws MessagingException, IOException {
        checkIfMailBoxExists(mailBox.getEmailAddress(), mailBox.getUser().getLogin());
        String domainPart = extractEmailDomain(mailBox.getEmailAddress());
        EmailConfiguration emailConfig = findEmailConfiguration(domainPart);

        mailBox.setEmailConfiguration(emailConfig);
        if (emailConfig == EmailConfiguration.GMAIL) {
            getAccessToken(mailBox);
        }

        validateCredentials(mailBox);
        mailBoxRepository.save(mailBox);
        return mailBox;
    }
```

Рисунок 10 Клас EmailService

Цей клас реалізує функціонал для управління електронною поштою в системі. Він відповідає за надсилання електронних листів, додавання конфігурацій поштових скриньок, автоматичне оновлення токенів доступу для Gmail, а також забезпечення перевірки облікових даних користувачів. Клас інтегрується з поштовими сервісами, використовуючи різні стратегії відправлення, і підтримує обробку поштових скриньок із різними доменами.

GmailSendStrategy:

```java
@Service("gmail")
public class GmailSendStrategy implements SendStrategy {
    1 usage
    private static final String CONTENT_TYPE_HTML = "text/html";
    1 usage
    private static final String XOAUTH2_AUTH_FORMAT = "user=%s\u0001auth=Bearer %s\u0001\u0001";
    2 usages
    private static final String SMTP_HOST = "smtp.gmail.com";

    8 usages
    private final Properties props;

    no usages  Dima Artamonov
    public GmailSendStrategy(
            @Qualifier("gmail-properties") Properties props) {
        this.props = props;
    }

    1 usage  Dima Artamonov
    @Override
    public Email sendWithStrategyEmail(Email email, MailBox mailBox) {
        Session session = createSession();
        session.setDebug(true);

        try {
            MimeMessage msg = createMessage(session, email, mailBox);
            sendEmailWithOAuth2(session, email, mailBox, msg);
            email.setSentAt(LocalDateTime.now());
            return email;
```

```java
    1 usage  Dima Artamonov
    private void sendEmailWithOAuth2(Session session, Email email, MailBox mailBox, MimeMessage msg)
            throws MessagingException {
        try (SMTPTransport transport = new SMTPTransport(session,  urlname: null)) {
            String authString = String.format(XOAUTH2_AUTH_FORMAT, email.getSenderEmail(),
                    mailBox.getAccessSmtp());
            String encodedAuthString =
                    Base64.getEncoder().encodeToString(authString.getBytes(StandardCharsets.UTF_8));

            transport.connect(SMTP_HOST, email.getSenderEmail(),  password: null);
            transport.issueCommand( cmd: "AUTH XOAUTH2 " + encodedAuthString,  expect: 235);
            transport.sendMessage(msg, msg.getAllRecipients());
        }
    }


    1 usage  Dima Artamonov
    private void configureXOAuth2() {
        props.put("mail.smtp.auth.mechanisms", "XOAUTH2");
        props.put("mail.debug.auth", "true");
        props.put("mail.smtp.host", SMTP_HOST);
    }
```

Рисунок 11 Клас GmailSendStrategy

Цей клас реалізує стратегію відправлення електронних листів через Gmail, використовуючи протокол OAuth2 для автентифікації. Він відповідає за створення та надсилання повідомлень, перевірку облікових даних користувача

Gmail, а також налаштування необхідних властивостей SMTP для забезпечення безпечного з'єднання. Клас інтегрується з поштовими серверами Gmail і забезпечує підтримку HTML-контенту для повідомлень.

UkrnetSendStrategy:

```java
@Service("ukr")
public class UkrNetSendStrategy implements SendStrategy {
    1 usage
    private static final String CONTENT_TYPE_HTML = "text/html";
    1 usage
    private static final String PROTOCOL_SMTP = "smtp";
    2 usages
    private final Properties props;

    no usages  Dima Artamonov
    public UkrNetSendStrategy(
            @Qualifier("ukr-properties") Properties props) {
        this.props = props;
    }

    1 usage  Dima Artamonov
    @Override
    public Email sendWithStrategyEmail(Email email, MailBox mailBox) {
        try {
            Session session = createSession(mailBox);
            MimeMessage msg = createMessage(session, email, mailBox);
            Transport.send(msg);
            email.setSentAt(LocalDateTime.now());
            return email;

        } catch (MessagingException | IOException e) {
            throw new SendException("Failed to send email: " + e.getMessage());
        }
    }
```

Рисунок 12 Клас UkrnetSendStrategy

Цей клас реалізує стратегію відправлення електронних листів через сервіс Ukr.net. Він забезпечує створення та надсилання повідомлень, перевірку облікових даних поштової скриньки користувача, а також налаштування з'єднання з сервером SMTP Ukr.net. Клас підтримує автентифікацію з використанням логіна та пароля та дозволяє надсилати HTML-контент у повідомленнях.

DraftServiceImpl:

```java
@RequiredArgsConstructor
public class DraftServiceImpl implements DraftService {
    1 usage
    private final List<Properties> listOfImapProperties;
    1 usage
    private final MailBoxRepository mailBoxRepository;
    1 usage
    private final ImapService imapService;
    2 usages
    private Properties imapProperties;

    1 usage  Dima Artamonov
    @Override
    public void saveEmailAsDraft(Email email, String login) throws Exception {
        var mailbox =
                mailBoxRepository.findByEmailAddressAndUserLogin(email.getSenderEmail(), login)
                        .orElseThrow(
                                () -> new InvalidEmailReaderException(
                                        "This email does not exist in your mailboxes.")
                        );
        var imapConfig = mailbox.getEmailConfiguration();
        setProperties(imapConfig.getImapHost());
        Session session = Session.getInstance(imapProperties);
        var store = imapService.getImapStore(email.getSenderEmail(), login);

        var folders = store.getDefaultFolder().list( pattern: "*");
        var draftFolder = Arrays.stream(folders) Stream<Folder>
                .filter(folder -> {
                    System.out.println(folder.getFullName().toLowerCase());
                    System.out.println(folder.getFullName().toLowerCase().contains("draft"));
                    return folder.getName().toLowerCase().contains("draft");

        var mimeMessage = new MimeMessage(session);
        mimeMessage.setFrom(new InternetAddress(email.getSenderEmail()));
        mimeMessage.setRecipients(Message.RecipientType.TO, email.getRecipientEmail());
        mimeMessage.setSubject(email.getSubject());
        mimeMessage.setContent(email.getBody(),  type: "text/html");

        Message[] messages = {mimeMessage};
        draftFolder.appendMessages(messages);
        draftFolder.close();
    }

    1 usage  Dima Artamonov
    private void setProperties(String domainName) {
        this.imapProperties = listOfImapProperties.stream() Stream<Properties>
                .filter(properties -> {
                    String host = properties.getProperty("mail.imap.host");
                    return host != null && host.contains(domainName);
                })
                .findFirst() Optional<Properties>
                .orElseThrow(() -> new PropertiesNotFoundException(
                        "No matching Properties bean found for domain: " + domainName));
    }

}
```

Рисунок 13 Клас DraftServiceImpl

Цей клас відповідає за збереження електронних листів у папку чернеток (Drafts) на поштовому сервері за допомогою протоколу IMAP. Він використовує

властивості сервера IMAP для встановлення з'єднання, шукає відповідну папку чернеток у скриньці користувача та додає створене повідомлення до цієї папки. Клас також забезпечує перевірку наявності поштової скриньки користувача та правильності конфігурацій IMAP.

ImapServiceImpl:

```
@RequiredArgsConstructor
public class ImapServiceImpl implements ImapService {
    1 usage
    private final List<Properties> listOfImapProperties;
    3 usages
    private final MailBoxRepository mailBoxRepository;
    2 usages
    private Properties imapProperties;

    no usages  Dima Artamonov
    public static SearchTerm getMessagesSearchTerm() {
        Date yesterdayDate = new Date(new Date().getTime() - (1000L * 60 * 60 * 24 * 7));
        return new ReceivedDateTerm(ComparisonTerm.GT, yesterdayDate);
    }

    1 usage  Dima Artamonov
    private static FetchProfile getFetchProfile() {
        FetchProfile fetchProfile = new FetchProfile();
        fetchProfile.add(UIDFolder.FetchProfileItem.ENVELOPE);
        fetchProfile.add(UIDFolder.FetchProfileItem.CONTENT_INFO);
        fetchProfile.add("X-mailer");
        return fetchProfile;
    }

    2 usages  Dima Artamonov
    private static void closeFolder(Folder folder) {
        if (folder != null && folder.isOpen()) {
            try {
                folder.close( expunge: true);
            } catch (MessagingException e) {
                e.printStackTrace();
```

```
    @Override
    public Message[] getEmails(String account, String folderName, String login) throws Exception {
        isUserAllowedToReadEmail(login, account);
        Store store = getImapStore(account, login);
        Folder folder = getFolderFromStore(store, folderName, Folder.READ_ONLY);
        int messageCount = folder.getMessageCount();
        if (messageCount == 0) {
            return new Message[0];
        }
        int start = Math.max(1, messageCount - 30);
        Message[] messages = folder.getMessages(start, messageCount);
        folder.fetch(messages, getFetchProfile());
        closeFolder(folder);
        closeStore(store);
        return messages;
    }

    1 usage  Dima Artamonov
    private Message getEmail(String account, String folderName, int msgnum, String login)
            throws Exception {
        Store store = getImapStore(account, login);
        Folder folder = getFolderFromStore(store, folderName, Folder.READ_WRITE);
        var message = folder.getMessage(msgnum);
        message.setFlag(Flags.Flag.SEEN,  set: true);
        return message;
    }
```

Рисунок 14 Клас ImapServiceImpl

```
@Override
public Store getImapStore(String account, String login) throws Exception {
    var mailbox = mailBoxRepository.findByEmailAddressAndUserLogin(account, login).get();
    var imapConfig = mailbox.getEmailConfiguration();
    setProperties(imapConfig.getImapHost());
    Session session = Session.getInstance(imapProperties);
    Store store = session.getStore( protocol: "imap");
    try {
        store.connect(imapConfig.getImapHost(), account, mailbox.getAccessSmtp());
    } catch (Exception e) {
        throw new ReadException(e.getMessage());
    }
    return store;
}

1 usage  Dima Artamonov
private void setProperties(String domainName) {
    this.imapProperties = listOfImapProperties.stream() Stream<Properties>
            .filter(properties -> {
                String host = properties.getProperty("mail.imap.host");
                return host != null && host.contains(domainName);
            })
            .findFirst() Optional<Properties>
            .orElseThrow(() -> new PropertiesNotFoundException(
                    "No matching Properties bean found for domain: " + domainName));
}
```

Рисунок 15 Клас ImapServiceImpl

Цей клас реалізує функціональність для роботи з IMAP-поштовими сервісами. Він дозволяє підключатися до поштових скриньок користувачів, отримувати доступ до папок і повідомлень, переглядати, створювати, переміщувати та читати електронні листи. Також забезпечується обробка помилок, перевірка прав доступу та управління конфігураціями IMAP для поштових сервісів. Клас використовується для інтеграції з поштовими серверами та забезпечує зручний доступ до поштових даних через відповідні методи.

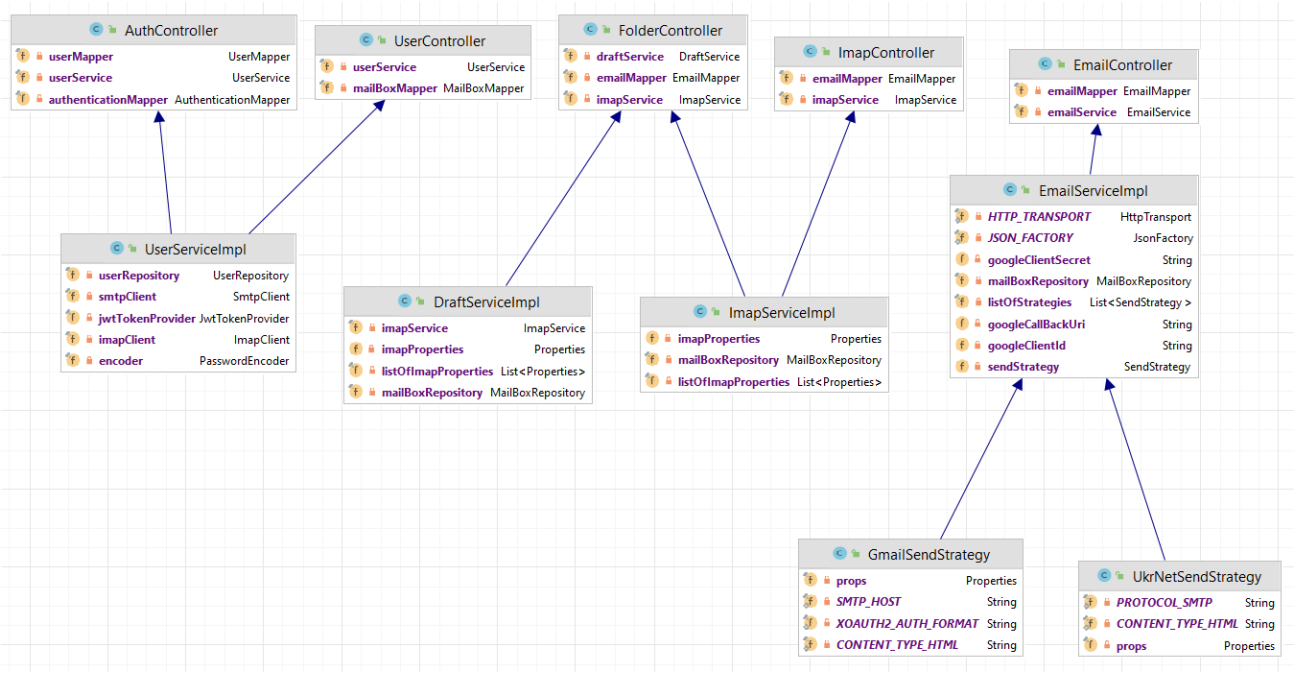


Рисунок 16. Діаграма класів архітектури «SOA».

На діаграмі представлена структура взаємодії між компонентами системи, що пов'язана з обробкою користувачів, роботою з поштовими сервісами та управлінням електронною поштою. Опис основних компонентів:

1. **Контролери:**
    - **AuthController:** Обробляє запити для аутентифікації користувачів, взаємодіючи з UserMapper, UserService та AuthenticationMapper.
    - **UserController:** Виконує функції, пов'язані з управлінням користувачами, використовуючи сервіси UserService і MailBoxMapper.
    - **FolderController:** Відповідає за роботу з папками електронної пошти через сервіси DraftService та ImapService.
    - **ImapController:** Обробляє запити, пов'язані з IMAP-поштовими сервісами, використовуючи ImapService.
    - **EmailController:** Обробляє запити для роботи з електронною поштою через сервіси EmailMapper та EmailService.
2. **Сервіси:**
    - **UserServiceImpl:** Реалізація логіки для управління користувачами. Залежить від репозиторіїв (UserRepository), клієнтів (SmtpClient, ImapClient) та інших сервісів, таких як JwtTokenProvider.

- **DraftServiceImpl:** Реалізує функції для збереження листів у чернетки, використовуючи ImapService, MailBoxRepository та IMAP-налаштування.
- **ImapServiceImpl:** Відповідає за взаємодію з IMAP-серверами для доступу до поштових даних, управління папками та обробку повідомлень.
- **EmailServiceImpl:** Реалізує логіку роботи з електронними листами, підтримуючи кілька стратегій відправки (SendStrategy) через різні сервіси, такі як GmailSendStrategy і UkrNetSendStrategy.
- **GmailSendStrategy:** Стратегія для роботи з поштою Gmail, використовує специфічні налаштування (SMTP_HOST, XOAUTH2_AUTH_FORMAT).
- **UkrNetSendStrategy:** Стратегія для роботи з UkrNet, що включає SMTP-налаштування для відправки повідомлень.

Архітектура SOA вирішує низку проблем, а саме:

- **Розподіл функціональності**: Кожен сервіс у системі відповідає за конкретну задачу, наприклад, надсилання листів, синхронізацію папок або роботу з вкладеннями.
- **Просте масштабування**: Компоненти клієнта, такі як сервіси для роботи з SMTP чи IMAP, можуть функціонувати незалежно, що дозволяє підвищувати продуктивність окремих модулів без змін у загальній системі.
- **Гнучке розширення**: Додавання нових функцій, наприклад, категоризації листів або автоматичного налаштування акаунтів, відбувається через окремі сервіси, не впливаючи на роботу решти системи.

Переваги використання SOA у поштовому клієнті:

1. **Модульність**:

   Кожен сервіс, наприклад, обробка вкладень чи збереження чернеток, ізольований, що спрощує його тестування, обслуговування та оновлення.

2. **Повторне використання**:

   Сервіси, такі як відправка листів або синхронізація папок, можуть використовуватися в різних частинах програми або навіть у різних поштових клієнтах.

3. **Масштабованість**:

Програма може легко адаптуватися до збільшення кількості користувачів або обсягу поштових скриньок завдяки незалежному масштабуванню таких сервісів, як обробка вхідних повідомлень.

4. **Незалежність**:

   Комунікація між сервісами здійснюється через API, що дозволяє їх окремо розробляти, тестувати та оновлювати, наприклад, змінювати логіку роботи з IMAP, не зачіпаючи інші модулі.

5. **Гнучкість**:

   Нові функції, такі як інтеграція з календарем чи фільтрація спаму, можуть додаватися без впливу на існуючу логіку роботи клієнта.

Посиланні на код проекту: https://github.com/qjsoq/Email-microservices

Висновок: У даній лабораторній роботі реалізовано систему поштового клієнта з використанням архітектури SOA. Логіка роботи була розділена на сервіси для управління поштовими акаунтами, обробки вхідних та вихідних повідомлень, роботи з протоколами SMTP/IMAP, синхронізації папок, обробки вкладень, категоризації листів та налаштування фільтрів. Це забезпечило гнучкість, зручність у розширенні функціоналу та ефективність використання ресурсів системи.